

7. Container

[문제] v에서 3을 제거하라.

```
/*vector<int> v{ 1, 2, 3, 4, 5 };
```

```
for (int n : v)
```

```
    cout << n << ' ';
```

```
cout << endl;*/
```

```
//vector<int>::iterator new_end = remove(v.begin(), v.end(), 3);
```

```
//v.erase(new_end, v.end());      //new_end ~ end() 범위를 쓰레기값이라고 바꾼다.
```

```
//remove 함수가 컨테이너 내부의 값을 아예 바꿔버리는 건 vector 컨테이너의 권한을 무시하는 행동이다.
```

```
//때문에 remove함수는 4, 5를 앞으로 옮기고 원래 5가 있던 자리를 데이터의 마지막 위치라고 알려주는 것이다.
```

```
/*v.erase(remove(v.begin(), v.end(), 3), v.end());*/
```

```
//C++ 20에서는
```

```
//erase(v, 3);    //기존 C++의 문법을 파괴해서 다들 싫어하는 듯 하다...일단 전역 함수라는 명칭이 있다.
```

```
/*cout << "현재 v의 용량 - " << v.capacity() << endl;
```

```
cout << "현재 v의 원소 개수 - " << v.size() << endl;*/
```

```
//remove할 때에는 반드시 erase도 같이 해야한다. erase-remove idiom(숙어)이라고 불리는 패턴이다.
```

[문제] v에서 길이가 2글자인 element를 삭제하라.

```
//vector<XString> v{ "1", "22", "4444", "5555", "33", "77"};
```

```
/*auto new_end = remove_if(v.begin(), v.end(), [](const XString& x) {
```

```
    return x.size() == 2;
```

```
});
```

```
v.erase(new_end, v.end());*/
```

```
/*auto Length2Comp = [](const XString& x) {
```

```
    return x.size() == 2;
```

```
};
```

```
erase_if(v, Length2Comp);
```

[문제] 컨테이너를 길이 기준 오름차순으로 정렬하라

```
/*sort(cont.begin(), cont.end(), [](const XString& a, const XString& b) {
```

```
    return a.size() < b.size();
```

```
});*/
```

//list는 연결리스트 방식으로 데이터를 저장하기 때문에 -연산자를 쓸 수 없어서 sort 함수를 그냥 쓸 수 없다. list 클래스의 sort 멤버 함수를 써야한다.

```
/*lookup = true;
```

```
cout << "list 컨테이너 정렬" << endl;
```

```
cont.sort([](const XString& a, const XString& b) {
```

```
    return a.size() < b.size();
```

```
});
```

```
cout << "list 컨테이너 정렬 끝" << endl;
```

```
cout << "vector 컨테이너 정렬" << endl;
```

```
sort(cont2.begin(), cont2.end(), [](const XString& a, const XString& b) {
```

```
    return a.size() < b.size();
```

```
});
```

```
cout << "vector 컨테이너 정렬 끝" << endl;
```

[문제] 강의 저장 파일을 list<XString>에 저장하라.

```
//길이 오름차순으로 정렬하라.
```

```
//출력하라.
```

```
//ifstream in{ "main.cpp" };
```

```
//if (not in) return 20260429;
```

```
//ifstream in2{ "main.cpp" };
```

```
//if (not in) return 20260429;
```

```
//
```

```
//list<XString> words{ istream_iterator<XString>{in}, {} }; //벡터는 이렇게 불러오면 안된다. list만 이렇게...
```

```
//vector<XString> cont2{};
```

```
//cont2.reserve(2'0000);
```

```
//cont2.assign(istream_iterator<XString>{in2}, {}); //벡터는 이렇게 불러와야 한다.
```

```
////list<XString> cont{ cont2.begin(), cont2.end() }; //벡터에서 리스트로 옮기는 방법
```

```
////cont2.clear();
```

```
//auto LengthComp = [](const XString& a, const XString& b) {
```

```
//     return a.size() < b.size();
```

```
//     };
```

//이동 생성은 list가 더 하지 않지만 어쨌선지 정렬이 Vector가 더 빠르다.

//간단히 말하면 list는 데이터를 액세스 하기 위해 반드시 Sequential하게 접근해야 하기 때문에 느리기 때문이다.

[문제] 이번엔 사전순으로 정렬하라. Lexicographical comparison

```
//auto LexComp = [](const XString& a, const XString& b) {
```

```
//     return lexicographical_compare(a.data(), a.data() + a.size(), b.data(), b.data() + b.size()); // 문자열을  
사전식으로 비교하는 함수
```

```
//     };
```

```
//words.sort(LexComp);
```

```
//sort(cont2.begin(), cont2.end(), LexComp);
```

```
//lookup = true;
```

```
//for (const XString& xs : words)
```

```
//     //xs.show();
```

```
//     cout << xs << endl;
```

```
//lookup = false;
```

```
//cout << "전체 단어 개수 - " << cont2.size() << endl;
```

```
////adjacent_find(); //인접한 원소 중에서 같은 원소가 있는지 찾아주는 함수.
```

```
//words.unique();
```

```
//auto newEnd = unique(cont2.begin(), cont2.end());
```

```
//cont2.erase(newEnd, cont2.end()); //Vector에서는 이렇게 해야지 제대로 중복 삭제가 된다.
```

```
//for (const XString& xs : cont2)
```

```
//     cout << xs << endl;
```

```
//cout << "서로 다른 단어의 개수는 몇 개인가? - " << cont2.size() << endl;
```

9. Iterator

```
//XString xs{ "the quick brown fox jumps over the lazy dog" };    //영문에서 모든 알파벳을 포함하는 문장.  
//sort(xs.begin(), xs.end());  
  
/////range-for문을 사용하려면 반복자 인터페이스가 필요하다.  
//for (char c : xs)  
//    cout << c << '-';  
//cout << endl;
```

10. Iterator2